

Week 2 AI Learning Plan

LangChain + Gemini — 5 days Mon-Fri — ~2 hrs/day — langchain-google-genai

- LLM layer
- NLP layer
- GenAI layer
- YouTube
- Blog
- Course

Week 1 → Week 2 connection
Every LangChain concept this week replaces something you built manually in Week 1. Day 2 memory replaces your Day 6 history loop. Day 1 chains replace your Day 1 raw API calls. You already understand what is happening underneath — now LangChain automates it.

WEEK 2 OVERVIEW		
Day 1	LangChain setup + chains + prompt templates First chain with Gemini	~2 hrs
Day 2	Memory types + Gemini integration ConversationChain replaces Day 6	~2 hrs
Day 3	LangChain agents + tools LLM that decides what to do	~2 hrs
Day 4	Document loaders + text splitters RAG preparation	~1.5 hrs
Day 5	Week 2 combined project Full LangChain app	~2 hrs

LangChain + Gemini — Quick Reference Cheat Sheet

All Week 2 patterns in one place. Flip here when you forget syntax.

Setup — install + import (run once)

```
pip install langchain langchain-google-genai langchain-community

from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

llm = ChatGoogleGenerativeAI(model="gemini-2.0-flash", google_api_key="key")
```

Basic LCEL chain

```
prompt = ChatPromptTemplate.from_messages([
    ("system", "Answer in {style}.",),
    ("human", "{question}")
])

chain = prompt | llm | StrOutputParser()
result = chain.invoke({"style": "bullets", "question": "What is AI?"})
print(result) # plain string
```

ConversationChain with memory

```
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationChain

memory = ConversationBufferMemory()
conv = ConversationChain(llm=llm, memory=memory)

conv.predict(input="What is a transformer?")
```

Summary memory (saves tokens)

```
from langchain.memory import ConversationSummaryMemory

memory = ConversationSummaryMemory(llm=llm)
conv = ConversationChain(llm=llm, memory=memory)
# Old turns are summarised – token count stays low
```

Agent with @tool

```
from langchain.tools import tool
from langchain.agents import AgentExecutor, create_tool_calling_prompt

@tool
def calculator(expression: str) -> str:
    """Evaluate a math expression. Use for calculations."""
    try: return str(round(eval(expression), 4))
    except: return "Error"

agent_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant. Use the calculator tool for math."),
    ("human", "{question}")
])
```

Load PDF + split into chunks

```
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

pages = PyPDFLoader("doc.pdf").load()
chunks = RecursiveCharacterTextSplitter(
    chunk_size=500, chunk_overlap=50
).split_documents(pages)
print(f"{len(chunks)} chunks, avg {sum(len(c.page_content) for c in chunks)} tokens per chunk")
```

Store + query ChromaDB

```
from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langchain_community.vectorstores import Chroma

embeddings = GoogleGenerativeAIEmbeddings(model="models/gemini-embedding-001", google_api_key="key")
vectorstore = Chroma.from_documents(chunks, embeddings, persist_directory="./chroma_db")
results = vectorstore.similarity_search("your question", k=3)
for doc in results: print(doc.page_content[:200])
```

Week 2 notebook naming convention

```
ai_learning/week2/
  day1_chains.ipynb           day4_documents.ipynb
  day2_memory.ipynb          day5_combined_project.ipynb
  day3_agents.ipynb
```

Day 1 LangChain setup + chains + prompt templates | First chain with Gemini

~2 hrs

ORG CONNECTION: LCEL chains are how every org AI app structures its prompt → model → output pipeline

LLM layer

Theory — LangChain architecture and LCEL

TOPICS IN THIS SECTION:

LangChain

Pipe operator |

ChatPromptTemplate

LCEL

Runnable

StrOutputParser

TOPICS TO COMPLETE — check each as you understand it

- ☒ What LangChain is and why it exists as a framework
- ☒ What LCEL (LangChain Expression Language) is and why it replaced old chains
- ☒ How the pipe operator | connects prompt → llm → parser
- ☒ What a Runnable is and why every LangChain component is one
- ☒ Difference between old chain classes and new LCEL syntax

LangChain is a framework that wraps LLMs and adds tooling on top. LCEL — LangChain Expression Language — is the modern way to build in LangChain. It uses the pipe operator | to connect components: prompt | llm | parser. Each component is a Runnable — they chain together to form a pipeline. This replaced the old verbose chain classes and is now the standard.

FOCUS ON LCEL pipe syntax and how components connect | Why StrOutputParser converts AIMessage to string | ChatPromptTemplate vs PromptTemplate

SKIP FOR NOW LangChain Hub, LangServe, batch/stream for now

RESOURCES

- ☐ **YouTube** Patrick Loeber — LangChain Crash Course for Beginners (30 min)
- ☐ **YouTube** Krish Naik — Complete LangChain Tutorial
- ☒ **Blog** LangChain LCEL official docs
- ☒ **Blog** LangChain Google GenAI integration

READ MORE — TOPIC REFERENCES

GeeksForGeeks
Introduction to LangChain

Javatpoint
LangChain Tutorial

Analytics Vidhya
LangChain Powered Apps

Towards DS
Getting Started with LangChain

— Take a 10 min break before starting practice —

GenAI layer

Practice — First LangChain chain with Gemini

TOPICS IN THIS SECTION:

ChatGoogleGenerativeAI

LCEL chain

StrOutputParser

invoke()

PromptTemplate

chain.invoke()

TOPICS TO COMPLETE — check each as you finish the coding task

- ☒ pip install langchain langchain-google-genai langchain-community chromadb pypdf
- ☒ Create ChatGoogleGenerativeAI model object
- ☒ Call llm.invoke() and read response.content (not response.text)
- ☒ Build ChatPromptTemplate with system + human messages
- ☒ Add StrOutputParser and chain with pipe |
- ☒ Run chain with .invoke() using a dict of variables
- ☒ Compare output vs your Week 1 raw Gemini API call

WATCH BEFORE CODING

YouTube **CONCEPTS** Patrick Loeber — LangChain Crash Course (30 min)

Free Course **OPENAI** DeepLearning.AI — LangChain for LLM App Dev (free)

OFFICIAL DOCS — NEW SDK (always correct syntax)

NEW SDK

[LangChain LCEL quickstart](#)

NEW SDK

[ChatGoogleGenerativeAI reference](#)**COMMON ERRORS + QUICK FIXES****ModuleNotFoundError: langchain_google_genai** → Run: `pip install langchain-google-genai --upgrade`**AttributeError: response has no attribute text** → Use `response.content` not `response.text` — LangChain returns AIM**KeyError in ChatPromptTemplate** → Variable name in {braces} must exactly match key in `invoke()` d**Invalid model name error** → Use "gemini-2.0-flash" not "gemini-flash" or "gemini-pro"**Step 1** Install and setup

Run this once before anything else.

```
pip install langchain langchain-google-genai langchain-community
```

Step 2 First ChatGoogleGenerativeAI callNew cell. Notice: `response.content` not `response.text` — LangChain returns AIMessage objects.

```
from langchain_google_genai import ChatGoogleGenerativeAI

llm = ChatGoogleGenerativeAI(
    model="gemini-2.0-flash",
    google_api_key="your-key-here"
)

response = llm.invoke("What is LangChain in one sentence?")
print(response.content)      # use .content not .text
print(type(response))        # AIMessage – not a string
```

Note: Week 1 connection: this is the same Gemini model you used with the raw API. LangChain wraps it — same engine, different steering wheel.**Step 3** Build your first LCEL chainNew cell. The `|` operator connects components left to right.

```
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant. Answer in {style}.",),
    ("human", "{question}")
])
parser = StrOutputParser()

chain = prompt | llm | parser # LCEL pipe

result = chain.invoke({
    "style": "3 bullet points",
    "question": "What is machine learning?"
})
print(result)
print(type(result)) # str – parser converted AIMessage to string
```

Week 1 connection: The | pipe is the key LangChain concept. prompt formats your input, llm generates the response, parser converts AIMessage to a plain string

Step 4 Run chain with multiple inputs

New cell. One chain, different inputs — this is why templates matter.

```
topics = ["neural networks", "transformers", "LangChain"]
for topic in topics:
    result = chain.invoke({
        "style": "one sentence",
        "question": f"What is {topic}?"
    })
    print(f"{topic}:\n {result}\n")
```

Note: You just ran the same chain 3 times with different inputs. In Week 1, you would have written 3 separate API calls. This is the value of prompt templates.

End of Day 1 — mark each one when done

- ☒ LangChain installed and imported without errors
- ☒ ChatGoogleGenerativeAI works — I called .invoke() and got a response
- ☒ I understand why response.content not response.text in LangChain
- ☒ First LCEL chain runs: prompt | llm | parser
- ☒ I ran the chain with 3 different inputs using a loop
- ☒ I understand what the | pipe operator does in LCEL
- ☒ Notebook saved as week2_day1_chains.ipynb

NEXT: Day 2: Memory types — ConversationBufferMemory vs ConversationSummaryMemory. Rebuild your Week 1 Day 6 chatbot in 5 lines of LangChain code.

MY NOTES — write your key takeaway before moving to the next day

Takeaway: _____

Surprised by: _____

Question: _____

Day 2 Memory types + Gemini integration | ConversationChain replaces Day 6

~2 hrs

ORG CONNECTION: Buffer memory for short org chatbot sessions, summary memory for long support conversations — you choose based on session length

LLM layer **Theory — Memory types and when to use each**

TOPICS IN THIS SECTION:

Memory

ConversationSummaryMemory

Token cost

ConversationBufferMemory

ConversationChain

TOPICS TO COMPLETE — check each as you understand it

- ☒ ConversationBufferMemory — stores all messages, when to use it
- ☒ ConversationSummaryMemory — summarises old turns, token cost benefit
- ☒ Why memory type affects token cost in long conversations
- ☒ ConversationChain — how it wraps llm + memory together
- ☒ When buffer memory breaks down (long sessions) and summary helps

LangChain provides multiple memory types for different situations. Buffer memory keeps every message — simple but tokens grow with every turn. Summary memory summarises old turns to save tokens but loses exact detail. Entity memory tracks specific named things across the conversation. Choosing the right memory type is one of the first architecture decisions in any production chatbot.

FOCUS ON Token cost difference between memory types | When to switch from buffer to summary in org apps

SKIP FOR NOW VectorStoreRetrieverMemory, CombinedMemory, EntityMemory for now

RESOURCES

- ☐ **YouTube** Krish Naik — LangChain Memory Tutorial
- ☐ **Blog** LangChain Memory docs
- ☐ **Course** DeepLearning.AI — LangChain App Dev, Memory lesson (free)

READ MORE — TOPIC REFERENCES

GeeksForGeeks
Memory in LangChain

Towards DS
LangChain Memory for QA Systems

Analytics Vidhya
Memory in LangChain — AV guide

— Take a 10 min break before starting practice —

GenAI layer

Practice — ConversationChain replaces your Week 1 Day 6 chatbot

TOPICS IN THIS SECTION:

ConversationChain

chat memory

summary memory

predict()

buffer memory

TOPICS TO COMPLETE — check each as you finish the coding task

- ☒ Import ConversationBufferMemory and ConversationChain
- ☒ Build ConversationChain with Gemini llm
- ☒ Run 3 turns and verify memory works (ask "what did I just ask?")
- ☒ Inspect memory.chat_memory.messages to see stored history
- ☒ Swap to ConversationSummaryMemory and run same 3 turns
- ☒ Compare token count: buffer vs summary after 5 turns

WATCH BEFORE CODING

- YouTube** **OLD SDK** Krish Naik — LangChain Memory explained
- Free Course** **OPENAI** DeepLearning.AI — Memory lesson in LangChain course (free)

OFFICIAL DOCS — NEW SDK (always correct syntax)

NEW SDK LangChain ConversationChain docs

NEW SDK ConversationSummaryMemory reference

COMMON ERRORS + QUICK FIXES**ImportError: cannot import ConversationBufferMemory** → Run: `pip install langchain --upgrade`**chain.predict() returns wrong answer about history** → Make sure using `chain.predict(input=...)` not `llm.invoke()` — on**ConversationSummaryMemory TypeError: llm required** → Pass `llm=` to `ConversationSummaryMemory` — it needs the model to**Memory not resetting between test runs** → Create a new memory = `ConversationBufferMemory()` for each test**Step 1** ConversationBufferMemory + ConversationChain

New cell. Compare this with your Week 1 Day 6 chatbot — same result, fraction of the code.

```

from langchain_google_genai import ChatGoogleGenerativeAI
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationChain

llm = ChatGoogleGenerativeAI(
    model="gemini-2.0-flash",
    google_api_key="your-key-here"
)

memory = ConversationBufferMemory()
chain = ConversationChain(llm=llm, memory=memory)

response1 = chain.predict(input="What is a transformer in AI?")
response2 = chain.predict(input="How does attention work inside it?")
response3 = chain.predict(input="What were the two things I just asked about?")

print(response3) # Should correctly mention both topics
print(f"Messages in memory: {len(memory.chat_memory.messages)}")

```

Week 1 connection: Week 1 connection: your Day 6 chatbot needed 30+ lines with a manual history loop. This does the same thing in 6 lines.**Step 2** Inspect what memory stores

New cell. See exactly what LangChain sends to the API on every call.

```

print("Memory contents:")
for i, msg in enumerate(memory.chat_memory.messages):
    role = type(msg).__name__
    print(f"[{i+1}] {role}: {msg.content[:80]}...")

```

Note: This is the same history list you built manually on Day 6. LangChain just manages it for you automatically.**Step 3** ConversationSummaryMemory — save tokens on long conversations

New cell. Swap the memory type — notice the difference in stored content.

```
from langchain.memory import ConversationSummaryMemory

# Summary memory uses the LLM to summarise old turns
memory_summary = ConversationSummaryMemory(llm=llm)
chain_summary = ConversationChain(llm=llm, memory=memory_summary)

for q in ["What is neural network?", "How does backprop work?",
         "What is attention mechanism?", "What did I ask first?"]:
    r = chain_summary.predict(input=q)
    print(f"Q: {q[:40]}")
    print(f"A: {r[:150]}\n")

print("Summary stored:")
print(memory_summary.buffer)
```

Important: Summary memory costs more upfront (uses LLM to summarise) but saves tokens as conversations grow long. Use buffer for short sessions, summary for long ones.

End of Day 2 — mark each one when done

- ☒ ConversationChain with buffer memory correctly remembers previous questions
- ☒ I inspected memory.chat_memory.messages and saw the stored history
- ☒ ConversationSummaryMemory runs and shows a summary in memory_summary.buffer
- ☒ I understand when to use buffer vs summary memory
- ☒ I compared this with my Week 1 Day 6 chatbot and see the code reduction
- ☒ Notebook saved as week2_day2_memory.ipynb

NEXT: Day 3: LangChain agents — build an LLM that decides when to use tools. You will write a @tool function and watch the agent call it automatically.

MY NOTES — write your key takeaway before moving to the next day

Takeaway:

Surprised by:

Question:

Day 3

LangChain agents + tools | LLM that decides what to do

~2 hrs

ORG CONNECTION: Agents with tools are how your org AI assistant will handle tasks needing real-time data, calculations, or API calls

LLM layer

Theory — ReAct agent pattern — how agents think and act

TOPICS IN THIS SECTION:

- Agent
- Tool
- AgentExecutor
- ReAct pattern
- @tool decorator
- Reason-Act-Observe

TOPICS TO COMPLETE — check each as you understand it

- ☒ What makes something an agent vs a chain
- ☒ ReAct loop: Reason → Act → Observe → Repeat until done
- ☒ What a tool is and how the model decides when to call it
- ☒ The @tool decorator — how to define a tool in LangChain
- ☒ AgentExecutor — what it manages (loop, max iterations, verbose)
- ☒ When agents are better than chains (and when they are overkill)

An agent is an LLM that can decide what to do next instead of just following a fixed pipeline. The ReAct pattern (Reason + Act) has the model reason about what it needs, call a tool to get it, observe the result, and reason again. This loop continues until it has enough to answer. Every production AI agent uses some form of this pattern — it is the foundation of agentic AI.

FOCUS ON ReAct reasoning loop | Docstring in @tool is what the model reads to decide when to use it | verbose=True to see agent reasoning

SKIP FOR NOW Multi-agent systems, LangGraph, OpenAI functions agent for now

RESOURCES

- ☐ **YouTube** Krish Naik — LangChain Agents Tutorial
- ☐ **Blog** LangChain Agents concept docs
- ☐ **Blog** LangChain Tools docs

READ MORE — TOPIC REFERENCES**GeeksForGeeks**[LangChain Agents](#)**Analytics Vidhya**[LangChain Agents guide](#)**Towards DS**[Intro to LangChain Agents and Tools](#)

— Take a 10 min break before starting practice —

GenAI layer**Practice — Build an agent with a calculator tool****TOPICS IN THIS SECTION:**

- @tool**
- AgentExecutor**
- max_iterations**
- create_tool_calling_agent**
- verbose=True**

TOPICS TO COMPLETE — check each as you finish the coding task

- ☒ Define a @tool function with a proper docstring
- ☒ Create ChatPromptTemplate with agent_scratchpad placeholder
- ☒ Build agent with create_tool_calling_agent(llm, tools, prompt)
- ☒ Wrap in AgentExecutor with verbose=True
- ☒ Ask a question that requires the calculator — read the reasoning
- ☒ Add a second tool and test the agent choosing between them

WATCH BEFORE CODING

- YouTube** **OLD SDK** Krish Naik — LangChain Agents and Tools (30 min)
- Free Course** **OPENAI** DeepLearning.AI — LangChain Agents lesson (free)

OFFICIAL DOCS — NEW SDK (always correct syntax)**NEW SDK**[LangChain Tool calling agent docs](#)**NEW SDK**[LangChain @tool decorator reference](#)

COMMON ERRORS + QUICK FIXES**Agent uses wrong tool or ignores tool** → Improve the tool docstring — add "Use this when..." with clear**Max iterations exceeded error** → Add max_iterations=5 to AgentExecutor — prevents infinite loop**Tool output must be a string — TypeError** → Return str() from your @tool function always, even for numbers**agent_scratchpad KeyError** → Add ("placeholder", "{agent_scratchpad}") to your ChatPromptTe**Step 1** Define your first tool

New cell. The docstring is what the model reads to decide when to call this tool — write it clearly.

```

from langchain.tools import tool

@tool
def calculator(expression: str) -> str:
    """Evaluate a mathematical expression.
    Input must be a valid Python math expression like '15 * 847 / 100'.
    Use this whenever a calculation is needed.
    """
    try:
        result = eval(expression)
        return str(round(result, 4))
    except Exception as e:
        return f"Error: {str(e)}"

# Test the tool directly first
print(calculator.invoke({"expression": "15 * 847 / 100"}))
print(calculator.name)
print(calculator.description)

```

Note: The docstring is critical — the model reads it to decide IF and WHEN to call this tool. A vague docstring means the agent will misuse or ignore the tool.

Step 2 Build the agent and executor

New cell. verbose=True shows the agent reasoning step by step.

```

from langchain_google_genai import ChatGoogleGenerativeAI
from langchain.agents import AgentExecutor, create_tool_calling_agent
from langchain_core.prompts import ChatPromptTemplate

llm = ChatGoogleGenerativeAI(
    model="gemini-2.0-flash",
    google_api_key="your-key-here"
)

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant with access to tools."),
    ("human", "{input}"),
    ("placeholder", "{agent_scratchpad}")
])

tools = [calculator]
agent = create_tool_calling_agent(llm, tools, prompt)
executor = AgentExecutor(agent=agent, tools=tools, verbose=True, max_iterations=5)

```

Step 3 Run the agent and read the reasoning

New cell. Watch verbose output — you will see Reason → Act → Observe loop.

```
result = executor.invoke({
    "input": "What is 23% of 1500? Then add 47 to that result."
})
print("\nFinal answer:", result["output"])
```

Week 1 connection: verbose=True output shows you exactly how the agent thinks. It calls the calculator tool once for 23% of 1500, then again to add 47. This is the ReAct loop in action.

Step 4 Add a second tool — agent chooses which to use

New cell. Define a string tool so the agent has to decide which tool fits.

```
@tool
def word_counter(text: str) -> str:
    """Count the number of words in a given text.
    Use this when asked how many words something contains.
    """
    count = len(text.split())
    return f"{count} words"

executor2 = AgentExecutor(
    agent=create_tool_calling_agent(llm, [calculator, word_counter], prompt),
    tools=[calculator, word_counter], verbose=True
)

# Agent must decide which tool to use for each part
result = executor2.invoke({
    "input": "How many words are in this sentence: The quick brown fox. Also what is 8 t
})
print(result["output"])
```

Note: The agent reads both tool docstrings, decides the first question needs word_counter and the second needs calculator. This is how agents handle complex multi-step tasks.

End of Day 3 — mark each one when done

- ☒ I defined a @tool function with a clear docstring
- ☒ I tested the tool directly with tool.invoke() before adding to agent
- ☒ AgentExecutor runs and I can see the verbose ReAct reasoning
- ☒ The agent correctly called the calculator tool to solve the maths question
- ☒ I added a second tool and the agent correctly chose between them
- ☒ I understand that the docstring is what the model uses to pick the tool
- ☒ Notebook saved as week2_day3_agents.ipynb

NEXT: Day 4: Document loaders and text splitters — load a real PDF, chunk it, store in ChromaDB. This is the direct setup for your Week 3 RAG pipeline.

MY NOTES — write your key takeaway before moving to the next day

Takeaway:

Surprised by:

Question:

Day 4

Document loaders + text splitters | RAG preparation

~1.5 hrs

ORG CONNECTION: Chunk size is the first setting you will tune when your org RAG app gives wrong answers — now you know why

NLP layer

Theory — Chunking strategies — the most important RAG parameter

TOPICS IN THIS SECTION:

Chunking

chunk_size

chunk_overlap

RecursiveCharacterTextSplitter

Metadata

RAG prep

TOPICS TO COMPLETE — check each as you understand it

☒ Why documents need chunking before embedding

☒ chunk_size: what it controls and the tradeoffs at each size

☒ chunk_overlap: why it matters and typical values (10-15% of chunk_size)

☒ RecursiveCharacterTextSplitter vs CharacterTextSplitter — which to use

☒ How metadata (source, page number) travels with each chunk

☒ How chunk quality directly affects RAG answer quality

Before you can search documents with embeddings, you need to split them into chunks. Chunk size and overlap are the most important tuning parameters in any RAG system. Too small = chunks lose context and answers span multiple chunks. Too large = wastes tokens and dilutes similarity scores. Overlap prevents answers from being cut at chunk boundaries. These settings directly determine whether your RAG app gives correct answers.

FOCUS ON chunk_size vs chunk_overlap tradeoffs | RecursiveCharacterTextSplitter splits on paragraphs first then sentences then words

SKIP FOR NOW Semantic chunking, parent document retriever, markdown splitter for now

RESOURCES

☐ Blog

LangChain Text Splitters docs

☐ Blog

LangChain Document Loaders docs

☐ YouTube

Krish Naik — LangChain Document Loaders tutorial

READ MORE — TOPIC REFERENCES

GeeksForGeeks

Document Loaders in LangChain

GeeksForGeeks

Text Splitters in LangChain

Analytics Vidhya

Document Loaders and Text Splitters

— Take a 10 min break before starting practice —

GenAI layer

Practice — PyPDFLoader + RecursiveCharacterTextSplitter + ChromaDB

TOPICS IN THIS SECTION:

PyPDFLoader

RecursiveCharacterTextSplitter

ChromaDB

GoogleGenerativeAIEmbeddings

Chroma

TOPICS TO COMPLETE — check each as you finish the coding task

- ☒ pip install pypdf chromadb
- ☒ Load a PDF with PyPDFLoader — check page count and content
- ☒ Split with RecursiveCharacterTextSplitter (chunk_size=500, overlap=50)
- ☒ Print chunk count, average size, and first chunk content
- ☒ Try chunk_size=200 vs 500 vs 1000 and compare chunk counts
- ☒ Create GoogleGenerativeAIEmbeddings object
- ☒ Store chunks in ChromaDB with Chroma.from_documents()
- ☒ Run a similarity query on ChromaDB to test retrieval

WATCH BEFORE CODING

YouTube

OLD SDK

[Krish Naik — Document Loaders and Text Splitters](#)

YouTube

OLD SDK

[James Briggs — LangChain RAG prep tutorial](#)**OFFICIAL DOCS — NEW SDK (always correct syntax)**

NEW SDK

[PyPDFLoader reference](#)

NEW SDK

[ChromaDB with LangChain](#)**COMMON ERRORS + QUICK FIXES****FileNotFoundError: PDF not found** → Use absolute path or place PDF in same folder as notebook**pip install pypdf needed** → Run: pip install pypdf — separate from LangChain**ChromaDB dimension mismatch** → Do not mix embedding models — always use the same model for st**Chroma import error** → Run: pip install chromadb langchain-community**Step 1** Load a PDF

Download any PDF (company report, textbook chapter, Wikipedia article saved as PDF). Put it in the same folder as your notebook.

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("your_document.pdf") # change to your file
pages = loader.load()

print(f"Pages loaded: {len(pages)}")
print(f"First 300 chars: {pages[0].page_content[:300]}")
print(f"Metadata: {pages[0].metadata}")
```

Note: Metadata includes source filename and page number. This travels with every chunk so your RAG app can tell users which page the answer came from.

Step 2 Split into chunks and experiment with sizes

New cell. The chunk_size and overlap settings here directly affect your RAG quality in Week 3.

```

from langchain.text_splitter import RecursiveCharacterTextSplitter

# Try different sizes and compare
for chunk_size in [200, 500, 1000]:
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=int(chunk_size * 0.1) # 10% overlap
    )
    chunks = splitter.split_documents(pages)
    avg = sum(len(c.page_content) for c in chunks) // len(chunks)
    print(f"chunk_size={chunk_size}: {len(chunks)} chunks, avg {avg} chars")

```

Week 1 connection: Smaller chunks = more chunks = finer search but less context per chunk. Larger chunks = fewer chunks = more context but less precise search. 500 chars is a good starting point for most org documents.

Step 3 Store chunks in ChromaDB

New cell. This is the vector database you will query in Week 3.

```

from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langchain_community.vectorstores import Chroma

# Final chunks to store
splitter = RecursiveCharacterTextSplitter(chunk_size=500, chunk_overlap=50)
chunks = splitter.split_documents(pages)

# Create embeddings
embeddings = GoogleGenerativeAIEmbeddings(
    model="models/gemini-embedding-001",
    google_api_key="your-key-here"
)

# Store in ChromaDB (creates a local database)
vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory="./chroma_db"
)
print(f"Stored {len(chunks)} chunks in ChromaDB")

```

Note: Week 1 connection: GoogleGenerativeAIEmbeddings uses the same gemini-embedding-001 model you called directly on Day 4. LangChain just wraps it.

Step 4 Query ChromaDB — preview of Week 3 RAG

New cell. Ask a question and see which chunks are retrieved.

```

query = "What is the main topic of this document?"
results = vectorstore.similarity_search(query, k=3)

print(f"Query: {query}\n")
for i, doc in enumerate(results):
    print(f"Result {i+1} (page {doc.metadata.get('page', '?')})")
    print(f"  {doc.page_content[:200]}...\n    print()

```

Week 1 connection: This is exactly what Week 3 RAG does — query the vector store, get the most relevant chunks, pass them to the LLM to generate an answer. You just built the retrieval part.

End of Day 4 — mark each one when done

- ☒ PyPDFLoader loaded a real PDF and I checked page count and metadata
- ☒ RecursiveCharacterTextSplitter ran with 3 different chunk sizes and I compared results
- ☒ I understand why chunk_size=200 gives more chunks than chunk_size=1000
- ☒ Chunks are stored in ChromaDB without errors
- ☒ similarity_search() returned the 3 most relevant chunks for my query
- ☒ Notebook saved as week2_day4_documents.ipynb

NEXT: Day 5: Week 2 combined project — connect everything from Days 1-4 into one LangChain app: chain + memory + tool + document loading. Then compare it to your Week 1 Day 7 notebook.

MY NOTES — write your key takeaway before moving to the next day

Takeaway:

Surprised by:

Question:

Day 5

Week 2 combined project | Full LangChain app

~2 hrs

ORG CONNECTION: This notebook is your Week 2 deliverable — a full LangChain app ready to extend into a RAG system next week

LLM layer

Theory — LangSmith — observability for LLM apps

TOPICS IN THIS SECTION:

- ☒ LangSmith
- ☒ Observability
- ☒ Production monitoring
- ☒ Tracing
- ☒ Debugging chains

TOPICS TO COMPLETE — check each as you understand it

- ☒ What LangSmith traces — every chain step, prompt, response, latency
- ☒ Why observability matters when your chain has 5+ steps
- ☒ How to enable LangSmith with two environment variables
- ☒ What a trace looks like in the LangSmith dashboard
- ☒ Difference between debugging locally vs in production

LangSmith is LangChain's observability platform — it traces every LLM call, shows inputs and outputs at each step, measures latency, and helps debug complex chains. In production org projects, when a chain gives a wrong answer, LangSmith tells you exactly which step failed and why. Theory only today — you will use it when you productionise in Week 4.

FOCUS ON What LangSmith captures | Why you need it when things go wrong in Week 3

SKIP FOR NOW LangSmith evaluators, datasets, annotation queues — covered in Week 4

- RESOURCES
- ☐

Blog

 LangSmith quickstart docs
- ☐

YouTube

 LangSmith overview and demo

READ MORE — TOPIC REFERENCES

GeeksForGeeks

Introduction to LangSmith

Analytics Vidhya

LangSmith guide

— Take a 10 min break before starting practice —

GenAI layer

Practice — Week 2 combined project

TOPICS IN THIS SECTION:

LCEL chain

AgentExecutor

Combined project

ConversationChain

Chroma

TOPICS TO COMPLETE — check each as you finish the coding task

☒

 Build ChatGoogleGenerativeAI with LCEL chain + PromptTemplate

☒

 Add ConversationBufferMemory via ConversationChain

☒

 Add one @tool (calculator) with AgentExecutor

☒

 Load a PDF and split into ChromaDB

☒

 Run 3 multi-turn conversations and verify memory

☒

 Ask a question requiring the calculator tool

☒

 Query ChromaDB for a relevant chunk

☒

 Compare line count with Week 1 Day 7 notebook

WATCH BEFORE CODING

YouTube

CONCEPTS

Karpathy — State of GPT (rewatch for Week 2 context)

YouTube

CONCEPTS

Balaji Chippada — 99% Learn AI Wrong (Telugu)

OFFICIAL DOCS — NEW SDK (always correct syntax)

NEW SDK

Full LangChain Python reference

NEW SDK

LangChain + Google GenAI all integrations

COMMON ERRORS + QUICK FIXES

Section 3 agent conflicts with section 2 memory → Keep agent and ConversationChain as separate objects — they us

ChromaDB no such directory → Run Day 4 notebook first to create the chroma_db folder, then

ImportError on langchain_community → Run: pip install langchain-community --upgrade

Combined project runs but memory forgets → Each conv.predict() must use the same conv object — do not rec

Step 1 Week 2 combined project — all concepts in one notebook

Run each section separately first, then combine. This is your Week 2 deliverable.

```

from langchain_google_genai import ChatGoogleGenerativeAI, GoogleGenerativeAIEmbeddings
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser
from langchain.memory import ConversationBufferMemory
from langchain.chains import ConversationChain
from langchain.agents import AgentExecutor, create_tool_calling_agent
from langchain.tools import tool
from langchain_community.vectorstores import Chroma
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter

llm = ChatGoogleGenerativeAI(
    model="gemini-2.0-flash",
    google_api_key="your-key-here"
)

```

Step 2 Section 1: LCEL chain (Day 1)

New cell.

```

# ---- SECTION 1: LCEL CHAIN ----
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful enterprise assistant. Be concise."),
    ("human", "{question}")
])
chain = prompt | llm | StrOutputParser()

print("CHAIN OUTPUT:")
print(chain.invoke({"question": "What is LangChain in one sentence?"}))

```

Week 1 connection: Same result as Week 1 raw API, but now you have a reusable template. Swap the question, keep everything else.

Step 3 Section 2: Memory chatbot (Day 2)

New cell.

```

# ---- SECTION 2: MEMORY ----
memory = ConversationBufferMemory()
conv = ConversationChain(llm=llm, memory=memory)

conv.predict(input="What is a RAG pipeline?")
conv.predict(input="Why do we need vector databases for it?")
answer = conv.predict(input="What were the two topics I just asked about?")
print("\nMEMORY CHATBOT:")
print(answer)
print(f"Messages stored: {len(memory.chat_memory.messages)}")

```

Step 4 Section 3: Agent with tool (Day 3)

New cell.

```
# ---- SECTION 3: AGENT ----
@tool
def calculator(expression: str) -> str:
    """Evaluate a math expression. Use for any calculation."""
    try: return str(round(eval(expression), 4))
    except: return "Error"

agent_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant."),
    ("human", "{input}"),
    ("placeholder", "{agent_scratchpad}")
])
executor = AgentExecutor(
    agent=create_tool_calling_agent(llm, [calculator], agent_prompt),
    tools=[calculator], verbose=False
)
result = executor.invoke({"input": "What is 18% of 2500? Add 150 to that."})
print("\nAGENT OUTPUT:")
print(result["output"])
```

Step 5 Section 4: Document loading (Day 4)

New cell. Use the ChromaDB you built on Day 4 or create a new one.

```
# ---- SECTION 4: DOCUMENT RETRIEVAL ----
# Load from existing ChromaDB (Day 4) or create new:
embeddings = GoogleGenerativeAIEmbeddings(
    model="models/gemini-embedding-001",
    google_api_key="your-key-here"
)
vectorstore = Chroma(
    persist_directory="./chroma_db",
    embedding_function=embeddings
)

query = "What is the main topic of this document?"
results = vectorstore.similarity_search(query, k=2)
print("\nDOCUMENT RETRIEVAL:")
for i, doc in enumerate(results):
    print(f"Result {i+1}: {doc.page_content[:200]}...")
```

Note: Week 3 preview: next week you will connect this retrieval step directly to the LLM to generate answers — that is the full RAG pipeline.

Step 6 Week 1 vs Week 2 comparison

Run this last. Count the lines — this is the value LangChain adds.

```
# Week 1 Day 7: ~60 lines to build API calls + manual history + embeddings
# Week 2 Day 5: ~50 lines for chain + memory + agent + doc retrieval

week1_features = [
    "Raw Gemini API calls",
    "Manual history list (30+ lines)",
    "Direct embeddings API",
    "Cosine similarity function"
]

week2_features = [
    "LCEL chain (3 lines)",
    "ConversationChain memory (4 lines)",
    "Agent with tools (8 lines)",
    "ChromaDB vector search (4 lines)"
]

print("Week 1 manual approach:")
for f in week1_features: print(f" {f}")
print("\nWeek 2 LangChain approach:")
for f in week2_features: print(f" {f}")
```

Week 1 connection: Same capabilities, fraction of the code. Week 1 taught you what is happening. Week 2 gives you the tools to do it efficiently. Week 3 combines them into a full RAG app.

End of Day 5 — mark each one when done

- ☒ All 4 sections run without errors in the combined notebook
- ☒ Memory chatbot correctly answers "what did I ask about" with 2 topics
- ☒ Agent calls the calculator tool and gives the correct numerical answer
- ☒ ChromaDB returns relevant chunks for my query
- ☒ I compared Week 1 Day 7 vs Week 2 Day 5 and see the code reduction
- ☒ Notebook saved as week2_day5_combined_project.ipynb
- ☒ I am ready to start Week 3 — RAG pipeline

NEXT: Week 3: Build a full RAG app on your own org documents — vector search + retrieval chain + Q&A chatbot + Streamlit UI. This is your org project deliverable.

MY NOTES — write your key takeaway before moving to the next day

Takeaway: _____

Surprised by: _____

Question: _____

Week 2 Resources — Bookmark These

Official references and learning resources for LangChain Week 2 and beyond.

LangChain official docs — always current

- ☐ LangChain Python docs — python.langchain.com (main reference for all of Week 2)
- ☐ LangChain Google GenAI — python.langchain.com/docs/integrations/platforms/google
- ☐ LCEL concepts — python.langchain.com/docs/concepts/lcel
- ☐ LangChain agents — python.langchain.com/docs/concepts/agents

Free courses — Week 2 specific

- ☐ DeepLearning.AI — LangChain for LLM App Dev (Andrew Ng, free) — deeplearning.ai/short-courses
- ☐ DeepLearning.AI — LangChain Chat with your data / RAG (free) — deeplearning.ai/short-courses
- ☐ Hugging Face NLP course — huggingface.co/learn/nlp-course (use from Week 3 onwards)

YouTube — best LangChain channels

- ☐ Patrick Loeber (Python Engineer) — best for practical LangChain code — youtube.com/c/PythonEngineer
- ☐ Krish Naik — comprehensive LangChain playlist — youtube.com/@krishnaik06
- ☐ James Briggs — LangChain deep dives — youtube.com/@jamesbriggs

Reading references for Week 2 topics

- ☐ GFG LangChain series — geeksforgeeks.org/introduction-to-langchain
- ☐ Analytics Vidhya LangChain — analyticsvidhya.com (search LangChain)
- ☐ Towards Data Science LangChain — towardsdatascience.com (search LangChain)
- ☐ LangSmith docs — docs.smith.langchain.com (for Week 4 debugging)